

Task AB13. Vision

5 s 1024 MB

USS Enterprise տիեզերանավը (որի նոր կապիտանն է Sashka-ն) շրջում է տիեզերքում, որը ներկայացված է որպես անվերջ երկչափ ցանց (grid) սեկտորների: Ամեն սեկտորում (X, Y) ¹ կա փարոս՝ տեսանելիության ուժով $V_{X,Y}$: Երբ նավը գտնվում է այդ սեկտորում, այն օգտագործում է փարոսը՝ սկանավորելու բոլոր սեկտորները (X', Y') , որոնց համար կատարվում է $X - V_{X,Y} \leq X' \leq X + V_{X,Y}$ և $Y - V_{X,Y} \leq Y' \leq Y + V_{X,Y}$: Այսպիսով, տեսանելի սեկտորները կազմում են քառակուսի՝ կողմի երկարությամբ $2V_{X,Y} + 1$: Այդ սեկտորներից յուրաքանչյուրի համար տեսանելի է միայն այնտեղի փարոսի տեսանելիության ուժը, այսինքն՝ $V_{X',Y'}$ (քանի որ փարոսները գործում են և՛ որպես հաղորդիչներ, և՛ որպես ընդունիչներ): Սկանավորումից հետո նավը տեղափոխվում է այդ տեսանելի սեկտորներից որևէ մեկը, քանի որ փարոսները նաև հեռափոխման (teleportation) սարքեր են:

Ցավոք, փարոսներն ունեն մի կարևոր թերություն. նրանք կարող են շրջել սկանավորման X առանցքը, կամ Y առանցքը, կամ երկուսն էլ (այսինքն՝ կան 4 հնարավոր սկանավորման կողմնորոշումներ): Նկատենք, որ նավը օգտագործում է այս հնարավոր շրջված սկանը՝ որոշելու, թե որ սեկտոր տեղափոխվել, և տեղափոխումն էլ կատարվում է նույն կողմնորոշմամբ. օրինակ, եթե Y առանցքը շրջված է և նավը սկանի մեջ ընտրում է տեղափոխվել ձախ (նվազող Y), իրականում այն կտեղափոխվի աջ: Սա նշանակում է, որ նավը իսկապես կհայտնվի սկանի մեջ ընտրված սեկտորում:

Բացի այդ, նավը հիշողություն չունի, ուստի նրա որոշումը՝ թե ուր տեղափոխվել, պետք է կախված լինի միայն փարոսի ստեղծած սկանից: Բացի այդ, նավի ռազմավարությունը պետք է լինի դետերմինիստական — նույն սկանի դեպքում այն պետք է միշտ ընտրի նույն սեկտորը:

Նավը սկսում է անհայտ կոորդինատներից և պետք է միշտ (անկախ սկզբնական դիրքից և սկանների կողմնորոշումների հաջորդականությունից) կարողանա շարժվել դեպի աճող X և Y ուղղությունները՝ որքան ուզենա հեռու (կարելի է ասել, որ այն պետք է վերջապես հասնի մի սեկտորի (X, Y) , որի համար $X, Y \geq 10^{100}$): Սակայն, որպեսզի դա հնարավոր լինի, կարևոր են ինչպես նավի շարժման ռազմավարությունը, այնպես էլ տեսանելիության ուժերի դասավորությունը ամբողջ տիեզերքում:

Ահա այստեղ եք դուք ներգրավվում — դուք պետք է նախագծեք $N \times M$ չափի ուղղանկյուն նախշ P տեսանելիության ուժերի համար, որը անվերջ կկրկնվի տիեզերքում՝ $V_{X,Y} = P_{X \bmod N, Y \bmod M}$: Դուք նաև պետք է նախագծեք նավի շարժման ռազմավարությունը, այսինքն՝ ֆունկցիա, որը ստանում է փարոսի ստեղծած սկանը և որոշում է, թե որ սեկտոր տեղափոխվել հաջորդիվ:

Քանի որ ավելի մեծ տեսանելիության ուժ ունեցող փարոսները թանկ են, ձեր նպատակը վավեր լուծում կառուցելն է՝ միաժամանակ փորձելով նվազեցնել ձեր նախշում տեսանելիության ուժի միջին արժեքը, այսինքն՝ P -ի միջին արժեքը:

Քանի որ այս խնդիրը կարող է բավական բարդ թվալ, դուք որոշում եք սկսել պարզեցված տարբերակից՝ նույն խնդիրը, բայց 1D-ում: Այս պարզ տարբերակում

¹ X -ը տողի համարն է (աճում է դեպի ներքև ուղղությամբ), իսկ Y -ը սյունակի համարն է (աճում է դեպի աջ ուղղությամբ)։

XVII INTERNATIONAL ADVANCED TOURNAMENT IN INFORMATICS ONLINE 2026

մենք հեռացնում ենք X չափումը և օգտագործում միայն Y -ը: Այսպիսով, ձեր նախշը պարզապես երկարությամբ M հաջորդականություն է, և այն կրկնվում է այսպես՝ $V_Y = P_{Y \bmod M}$: Փարոսների ստեղծած սկանները նույնպես պարզապես երկարությամբ $2V_Y + 1$ հաջորդականություններ են (պարունակում են այն Y' սեկտորները, որոնց համար $Y - V_{X,Y} \leq Y' \leq Y + V_{X,Y}$): Այստեղ կա միայն 2 հնարավոր կողմնորոշում (սովորական կամ շրջված), և նավը պետք է կարողանա հասնել $Y \geq 10^{100}$:

Ձեր լուծումները 1D և 2D դեպքերի համար կգնահատվեն առանձին, և հնարավոր է մի դեպքում ստանաք միավորներ՝ առանց մյուսի համար վավեր լուծում ունենալու:

Իրականացման մանրամասներ

Դուք պետք է իրականացնեք 4 ֆունկցիա՝ `getVisionPattern1d`, `getMove1d`, `getVisionPattern2d` և `getMove2d`: Առաջին երկուսը օգտագործվում են 1D դեպքի համար, իսկ երկրորդ երկուսը՝ 2D-ի համար: Ամեն թեստում կկանչվի միայն այս գույգերից մեկը, բայց բոլոր 4-ը պետք է իրականացված լինեն (նույնիսկ եթե դրանցից մեկի իրականացումը դատարկ է), որպեսզի լուծումը համարվի վավեր (կոմպիլացվի):

```
std::vector<int> getVisionPattern1d()
```

Այս ֆունկցիան կկանչվի մեկ անգամ, եթե թեստը 1D դեպքի համար է: Այն պետք է վերադարձնի P հաջորդականությունը՝ երկարությամբ M — 1D տեսանելիության ուժի կրկնվող նախշը: M -ը և P -ի բոլոր տարրերը պետք է լինեն 1-ից 60 միջակայքում:

```
int getMove1d(std::vector<int> v)
```

Այս ֆունկցիան կկանչվի յուրաքանչյուր հնարավոր սկանի համար, եթե թեստը 1D դեպքի համար է: Այն ստանում է փարոսի ստեղծած սկանը՝ որպես երկարությամբ $2V + 1$ հաջորդականություն (որտեղ V -ն կենտրոնական սեկտորի փարոսի տեսանելիության ուժն է): Այն պետք է վերադարձնի, թե նավը որ սեկտոր պետք է տեղափոխվի՝ որպես ինդեքս T այս հաջորդականության մեջ, այնպես որ $0 \leq T < 2V + 1$:

```
std::vector<std::vector<int>> getVisionPattern2d()
```

Այս ֆունկցիան կկանչվի մեկ անգամ, եթե թեստը 2D դեպքի համար է: Այն պետք է վերադարձնի $N \times M$ չափի ուղղանկյուն աղյուսակ P — 2D տեսանելիության ուժի կրկնվող նախշը: N , M -ը և P -ի բոլոր տարրերը պետք է լինեն 1-ից 60 միջակայքում:

```
std::pair<int, int> getMove2d(std::vector<std::vector<int>> v)
```

Այս ֆունկցիան կկանչվի յուրաքանչյուր հնարավոր սկանի համար, եթե թեստը 2D դեպքի համար է: Այն ստանում է փարոսի ստեղծած սկանը՝ որպես $2V + 1$ չափի քառակուսի աղյուսակ (որտեղ V -ն կենտրոնական սեկտորի փարոսի տեսանելիության ուժն է): Այն պետք է վերադարձնի, թե նավը որ սեկտոր պետք է տեղափոխվի՝ որպես ինդեքսների գույգ S և T այս աղյուսակում, այնպես որ $0 \leq S, T < 2V + 1$:

Տրված չափականության D (1 կամ 2) դեպքում գնահատող համակարգը կստուգի, որ ձեր ծրագիրը կառուցում է վավեր նախշ — այսինքն՝ այն կատարում է ճիշտ հեռափոխման ընտրություններ բոլոր հնարավոր սկանների համար, և ձեր լուծումը աշխատում է անկախ սկզբնական կոորդինատներից և սկանների կողմնորոշումներից հաջորդականությունից:

Constraints

- $1 \leq D \leq 2$
- $1 \leq N, M, V_{X,Y}, V_Y \leq 60$

Subtasks and scoring

Subtask	Points	D	A^*
1	24	1	1.5
2	76	2	1.125

Your score for a given subtask (if your solution for it is correct) depends on the average vision strength in your pattern P for it. Let that be A and let the target for the subtask be A^* . Your score (fraction of points for the subtask) will be:

$$1 - \left(1 - \frac{A^* - 1}{\max(A, A^*) - 1}\right)^{0.8}$$

Sample pattern

Suppose your solution returns the following pattern with $N = 3$ and $M = 2$:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}$$

Now, let's examine the possible scans at sector $(0, 1)$, which has vision strength 2. There are 4 possible orientations (some of which produce the same scan on this pattern):

Orientation 0: No flips

$$\begin{bmatrix} 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \\ 2 & 1 & 2 & 1 & 2 \\ 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \end{bmatrix}$$

Orientation 1: Y-axis flipped

$$\begin{bmatrix} 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \\ 2 & 1 & 2 & 1 & 2 \\ 4 & 3 & 4 & 3 & 4 \\ 6 & 5 & 6 & 5 & 6 \end{bmatrix}$$

Orientation 2: X-axis flipped Orientation 3: Both axes flipped

$$\begin{bmatrix} 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \\ 2 & 1 & 2 & 1 & 2 \\ 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \end{bmatrix}$$

$$\begin{bmatrix} 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \\ 2 & 1 & 2 & 1 & 2 \\ 6 & 5 & 6 & 5 & 6 \\ 4 & 3 & 4 & 3 & 4 \end{bmatrix}$$

Since 0 and 1 are identical and since the ship is deterministic, only one call to `getMove2d` will be made for them. Suppose your solution returns the move $(0, 1)$ on this scan. On orientation 0 this would mean moving 1 to the left and 2 up, while on orientation 1 - moving 1 to the right and 2 up.

Sample grader

The sample grader's first input is D . After that it will do all calls to your function: first getting the pattern P and then caching all teleportation choices. After that, it will start a console interaction, asking for the starting coordinates of the ship. After that, it will print the current state: coordinates and "raw" (without flips) scan. Then, it will ask for the orientation to be used (0 for no flips, 1 to flip the Y axis, 2 to flip the X axis and 3 to flip both). After that, it will print the scan in the given orientation, as well as what move the ship choose. The ship will be moved and this will repeat. Notice that the sample grader will not automatically verify that your solution is correct.